# Breaking the Loop: AWARE is the New MAPE-K

Brell Sanwouo
brell.sanwouo@inria.fr
Univ. Lille, CNRS, Inria, UMR 9189
CRIStAL, F-59000
Lille, France

Clément Quinton
clement.quinton@univ-lille.fr
Univ. Lille, CNRS, Inria, UMR 9189
CRIStAL, F-59000
Lille, France

Paul Temple
paul.temple@inria.fr
Univ. Rennes, CNRS, Inria, IRISA
Rennes, France

## Abstract

Self-adaptive systems have traditionally relied on the MAPE-K loop. It consists of a centralized, reactive, and sequential loop for monitoring, analyzing, planning, and executing system adaptations. However, the increasing complexity and dynamic nature of modern systems have exposed the limitations of MAPE-K loops, including their lack of proactivity, scalability challenges, and difficulty integrating continuous learning or distributed decision-making. We introduce AWARE (Assess, Weigh, Act, Reflect, Enrich), a distributed, goal-driven framework that addresses these limitations. AWARE employs autonomous AI agents capable of proactive adaptation, collaboration, and continuous learning to enhance decision-making and system resilience. The modular design of our framework supports dynamic agent integration and optimized resource utilization, enabling seamless scalability and adaptability. AWARE not only anticipates changes and optimizes responses but also iteratively refines its strategies based on contextual insights. Through a comparison with MAPE-K and a real-world use case, we demonstrate how AWARE distributed intelligence redefines the capabilities of self-adaptive systems, offering a solution better aligned with the demands of complex real-world systems.

## CCS Concepts

• **Computer systems organization** → **Self-organizing autonomic computing**; • **Computing methodologies** → **Intelligent agents**.

## Keywords

Self-Adaptation, MAPE-K Loop, Intelligent Agents, Generative AI

**ACM Reference Format:**
Brell Sanwouo, Clément Quinton, and Paul Temple. 2025. Breaking the Loop: AWARE is the New MAPE-K. In *33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), June 23–28, 2025, Trondheim, Norway*. ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3696630.3728512

## 1 Introduction

Self-adaptive systems have long relied on the MAPE-K loop, introduced more than 20 years ago by Kephart et al. [20], as a fundamental framework for monitoring, analyzing, planning, and executing adaptations with the knowledge component serving as a centralized repository to store, manage, and share information that supports decision-making and coordination across the stages of the loop. Over the years, MAPE-K became a preferred choice in many areas for its simplicity and structure, such as cloud computing, the Internet of Things, or service-oriented architectures. However, modern IT landscapes have evolved considerably. Systems are now characterized by their increasing complexity and demands for real-time adaptability. Thus, they require an increased ability to perceive and understand the state of the system with the dynamics of their environment to act proactively, anticipate, and respond to changes before they become problematic.

The advent of intelligent agents [37], which integrate the fundamental principles of agents [42, 45] with the advanced capabilities of generative AI (GenAI) [3, 5, 11, 14, 21, 44], represents a significant evolution in adaptive computing. In the following, we consider an agent as an autonomous entity capable of perceiving its environment using sensors and acting on it using actuators to achieve defined goals or maximize a performance function [37]. In contrast with the centralized, sequential, and largely reactive nature of MAPE-K, these agents adopt a proactive approach to anticipate changes, generate efficient solutions, and adapt their strategies in real-time in complex and dynamic environments, making them particularly well suited to the demands of modern systems.

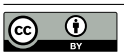
In this paper, we argue that MAPE-K is on the verge of becoming obsolete in the face of these technological changes and we propose AWARE as a new paradigm for self-adaptive systems. Unlike MAPE-K, AWARE leverages distributed intelligence to enable multiple intelligent agents to collaborate and make informed decisions at both the local and global levels. Leveraging information about system states and environmental conditions, AWARE improves adaptability and allows agents to anticipate scenarios, continuously learn, and refine their strategies for optimal results over time.

The remainder of the paper is as follows. Section 2 provides an overview of MAPE-K and its limitations. Section 3 presents AWARE, detailing its functionality, key advantages, and a motivational example comparing an adaptation performed using MAPE-K with one conducted using AWARE. Section 4 concludes the paper.

## 2 The MAPE-K Framework

### 2.1 Fundamentals

Self-adaptive systems are designed to dynamically adjust their behavior in response to changes in their environment or internal state. These systems are composed of two key components: the managing system, also called the adaptation logic, and the managed system, also known as the managed resources [22, 49]. The managing system functions as a control unit responsible for orchestrating the adaptation process of the managed system [22] operated by the runtime adaptation of the underlying software components. This




*FSE Companion '25, Trondheim, Norway*

ACM ISBN 979-8-4007-1276-0/2025/06
https://doi.org/10.1145/3696630.3728512

managing system is typically implemented using feedback control loops, among which the MAPE-K loop is one of the most widely recognized [6, 20, 48].

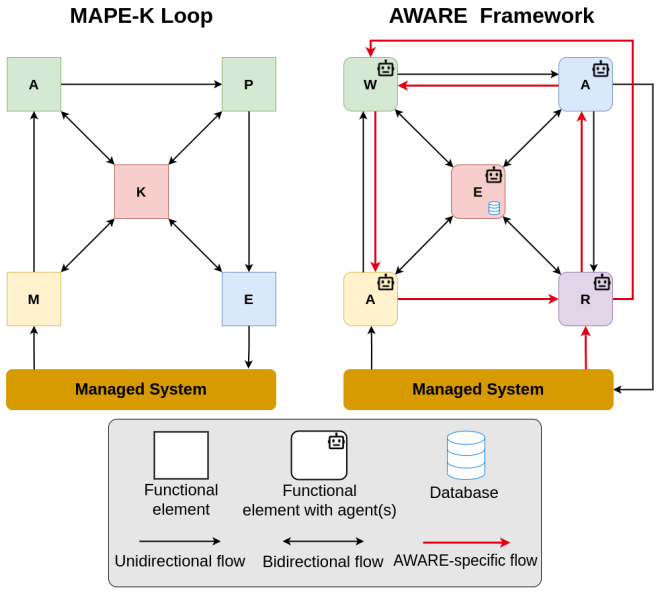



**Figure 1: MAPE-K Loop vs AWARE Framework. Functional elements' color shows how MAPE-K elements are mapped to the ones in AWARE. In particular, *Adapt* and *Plan* are merged into *Weigh* and we introduce *Reflect* for continuous learning. AWARE also enhances interconnections across components to improve information flow and decision-making capabilities**

Introduced more than two decades ago by Kephart *et al.* [20], MAPE-K is a cyclical framework to manage adaptation in self-adaptive systems. It comprises four key phases (*i.e.*, Monitor, Analyze, Plan, and Execute) that operate in conjunction with a shared knowledge base (K). The loop enables a system to monitor its internal and external states, analyze observations, formulate and execute decisions to optimize its performance or adapt to changes. As described by Weyns *et al.* [47], the *Monitor* phase detects changes within the managed system and its operational context. They are then reflected in execution models to assess the need for further analysis. The *Analyze* phase evaluates possible configurations for adaptation (adaptation options) and examines their feasibility relative to the goals of the system. The *Plan* phase selects the optimal adaptation option and generates an actionable plan to transition from the current managed system to the desired configuration. Finally, in the *Execute* phase, the planned actions are carried out, often automatically. Throughout this process, the knowledge base serves as a central repository, storing critical information that supports decision-making in all phases.

Over the years, MAPE-K loop have been extensively used to manage the adaptation process in various research and industrial domains, ranging from autonomic [1, 23, 30, 38] to Cloud computing systems [18, 19, 30, 31, 33], through IoT and cyber-physical environments [2, 33–35], network and telecommunication systems [10, 23, 41], cybersecurity [16, 17, 43], green computing [4, 29] and artificial intelligence [9, 24, 29].

## 2.2 Limitations

Despite its wide adoption, the MAPE-K framework exhibits several weaknesses that limit its implementation in modern, complex self-adaptive systems. First, during the *Monitoring* phase, only raw or minimally aggregated data are observed. At this stage, no meaningful contextual interpretation is applied, hindering the understanding of the root causes of the observed events. Secondly, its strictly sequential operation flow (Monitor → Analyze → Plan → Execute) makes it inherently reactive, limiting its ability to anticipate changes or explore future scenarios. This need for strict coordination between phases complicates the integration of new functionalities, reducing its adaptability. Moreover, the centralization of its knowledge base presents significant challenges in distributed or collaborative environments, restricting both its flexibility and scalability. Additionally, the MAPE-K loop lacks built-in continuous learning mechanisms, preventing it from refining decisions based on past experiences or incorporating new data. Addressing this limitation requires extending its design to support adaptive learning capabilities [24, 31].

Considering these limitations and the recent rise of GenAIs, in particular Large Language Models (LLMs), research work has attempted to integrate these models into various stages of the MAPE-K loop [7, 13, 15, 25–28, 36, 40]. Overall, this research work aims to improve performance and reduce uncertainty during adaptation. For example, Nakagawa *et al.* [32] propose a semi-automated process using GenAI to refine the *Plan* phase by generating goal models aligned with stakeholder requirements. Similarly, Sarda *et al.* [39] leverage LLMs to generate remediation scripts used during the *Plan* phase and automate the actions executed during the *Execute* phase.

However, while these approaches enhance specific phases within the MAPE-K loop, the loop as a whole remains inherently sequential, and the knowledge base remains centralized. Although LLMs are efficient at generating responses to specific prompts, managing them to ensure continuous learning remains challenging. Moreover, their ad-hoc operation limits proactive adaptation and continuous exploration of possibilities. Reliance on LLMs also introduces risks, such as biases, errors, or unpredictable behaviors, which can undermine the global system's reliability. Furthermore, the integration of GenAI into the MAPE-K loop is complicated by the vast diversity of options available. With a wide range of LLM models, countless application methods for different stages, and numerous parameters to configure, designing and fine-tuning the loop remains complex and error-prone.

# 3 AWARE: Agent-based Adaptation

## 3.1 Overview

The AWARE (Assess, Weigh, Act, Reflect, Enrich) framework relies on AI Agents, leveraging their learning, collaboration, and anticipation capabilities to promote a novel vision of self-adaptation. AI agents are autonomous entities capable of perceiving their environment, interpreting, predicting, and responding based on their training and input data [8]. As shown in Fig. 1, these capabilities enable

AWARE to go beyond the principles of the MAPE-K loop, offering distributed decision-making, proactive anticipation of change, and continuous adaptability through learning and scenario simulation. Unlike MAPE-K, AWARE's intelligent agents act autonomously and collaboratively, enabling faster, better-adapted responses to complex, dynamic environments. The AWARE framework relies on the five following AI agents.

**Assess [Observe and Understand]**. *Assess* agents go beyond only collecting raw data to detect changes in state or anomalies, as performed in the *Monitor* phase. They seek to perceive and understand the context to establish meaningful connections between the data and the context in which they are generated. *Assess* Agents interpret the data relying on a holistic and semantic view of the environment, the events and the interactions. These agents leverage AI-driven objective, with each observing according to the general or sub-objectives assigned to them, providing guidance on the relevance of the collected data. By the end of this phase, not only is the data collected, but one or more agents have understood both the relationship between the data itself and the relationship between the data and the system.

**Weigh [Evaluate and Decide]**. Each *Weigh* agent can formulate multiple adaptation options (*e.g.*, allocate more servers or limit certain functionalities, *etc.*) and evaluate them based on various criteria such as costs, risks, performance gains, and more. When objectives differ, negotiations can occur between agents. For example, the agent in charge of "performance" may propose deploying more servers, while the agent responsible of "budget" aims to reduce expenditure. Hypotheses are formulated and weighted according to their probability. *Weigh* agents evaluate several plans or options for adapting the system, taking into account multiple criteria while maintaining conformity with the overall objective. By the end of this phase, a decision (or set of actions) is made, that is considered the most appropriate to achieve the desired objective, given the current context.

**Act [Execute]**. *Act* Agents apply the decisions retrieved from the *Weigh* step. Multiple *Act* agents can perform simultaneous or coordinated actions *e.g.*, one agent reconfigures a component, another updates a firewall, and a third notifies an orchestrator. These actions can be highly localized or global, triggering distributed operations across the system. Additionally, a synchronization mechanism can be introduced to prevent conflicts between contradictory actions (*e.g.*, adjusting the same variable). Partially distributed executions are also possible, where local monitoring ensures each agent verifies that its modifications have been applied correctly.

**Reflect [Analyze and Learn]**. After executing the adaptations, *Reflect* agents assess the impact of their actions. This evaluation measures the quality of the decisions made and identifies any undesirable side effects. They compare the newly modified version of the system with the defined objectives. In the case of reinforcement agents, a reward mechanism can be used to calculate rewards or penalties. *Reflect* Agents also recognize when certain assumptions have been invalidated or when some actions have been more successful than expected. This stage serves as a foundation for rethinking future decisions, refining strategies, and anticipating potential challenges. By the end of this phase, concrete feedback is provided on the effectiveness and quality of the adaptation process.

| Criterion | MAPE-K | LLM MAPE-K | AWARE |
|---|---|---|---|
| Architecture | Centralized | Centralized | Distributed |
| Adaptation | Reactive | Reactive | Pro/re-active |
| Cont. learning | Absent | Partial | Complete |
| Anticipation | Nonexistent | Moderate | Complete |
| Evolutivity | Limited | Moderate | High |
| Collaboration | Absent | Absent | Strong |
| Knowledge | Centralized | Centralized | Distributed |
| Reliability | Medium | Variable | High |
| Implementation | Difficult | Difficult | Easy |

**Table 1: Comparison between MAPE-K, LLMs-enhanced MAPE-K, and AWARE.**

**Enrich [Update and Capitalize]**. The goal of this phase is to integrate into agents everything that has been observed and learned. *Enrich* agents can update their models or rules, and share this new knowledge with others. A synchronization mechanism may be employed, allowing certain insights to be shared with other agents dealing with similar situations. Additionally, a knowledge base can store global information for the entire system, making it increasingly intelligent with each iteration. This phase supports iterative or continuous fine-tuning, leading to the development of a more intelligent and high-performance system over time.

## 3.2 Advantages of AWARE

The AWARE approach is based on the principles of objective-driven AI [12], *i.e.*, typical agents (with clearly defined goals to reach) incorporating AI models to help them find appropriate adaptations. Table 1 summarizes the differences between AWARE and MAPE-K, whether classic or enhanced with LLMs.

The AWARE framework addresses the limitations of the MAPE-K loop by offering a more intelligent, distributed, and scalable approach based on continuous cooperation between agents. Contrary to the simple collection of raw data performed by the *Monitor* phase, the *Assess* agent integrates contextual perception, enabling the root causes of observed events to be understood and links between data to be established. In AWARE, agents do not only react to deviations: they anticipate the impact of their actions on goal achievement and take a proactive approach by experimenting with alternative strategies to assess their outcomes. AWARE's proactivity enables agents to anticipate future scenarios, run simulations, and act preventively, while MAPE-K, even when enhanced with LLMs, remains reactive and event-dependent to trigger adaptations. AWARE also ensures that decisions and learning processes in all agents are well-calibrated to improve goal satisfaction over time. Indeed, AWARE natively integrates continuous learning mechanisms thanks to the *Reflect* and *Enrich* phases, where agents immediately assess the impacts of their actions and update their models iteratively and collectively. This capability enables constant improvement of the system without the need for complex processes such as LLMs fine-tuning. Thus, by learning from each AI agent, AWARE can recognize patterns of efficiency and inefficiency, and make informed decisions when handling similar tasks in the future or adapting existing solutions to new requirements.

AWARE also addresses the limitations of MAPE-K's centralized architecture by adopting a distributed design, where each agent can independently collect, understand, and update its knowledge. Agents can act locally or collaborate with others, overcoming the challenges of integrating LLMs into MAPE-K and contributing to an enriched collective memory. Another key advantage is the scalability and modularity of AWARE. Unlike MAPE-K, which requires complex adjustments to integrate new functionalities or tackle novel problems, AWARE enables seamless addition or removal of specialized agents without updating the entire system. That is, the system can dynamically adjust the number of agents to accommodate changing workloads by adding new agents equipped with enriched knowledge (whether refined or more recent). This modularity also extends to the use of LLMs: rather than being rigidly integrated at a specific stage, LLMs can be used in a targeted way by agents as and when required, offering better resource utilization.

The role of humans in an AWARE system can vary according to the level of autonomy assigned to the agents. An AWARE system can be designed to operate fully autonomously or involve humans at various stages. For example, humans can define the overall strategy, choose or configure algorithms and learning rules, monitor and validate agent actions, analyze logs or reports generated *e.g.*, during the *Reflect* phase, refine adaptation parameters, objectives, or policy (*e.g.*, during the *Enrich* phase), or provide feedback based on their experience with the system.

In summary, AWARE goes beyond the limits of GenAI-enriched MAPE-K thanks to its flexibility, proactivity, continuous learning, and distributed approach, offering a solution better suited to the requirements of modern self-adaptive systems.

### 3.3 Motivation Example

This section compares the adaptation capabilities of the MAPE-K and AWARE frameworks using TeaStore as a use case. TeaStore is a microservice-based reference application specifically designed for benchmarking, modeling, and resource management research [46]. Designed as a real-world application that follows best practices, TeaStore operates as an online store for selling tea and related products. In our scenario, a promotional event increases user traffic, creating a sudden surge in active users. This increase puts a significant strain on certain microservices, such as store management and order processing.

*Adaptation with MAPE-K.* In the *Monitor* phase, the system collects key metrics from microservices, such as CPU utilization, memory usage, response time, and request rates. During the *Analyze* phase, the system evaluates these metrics and checks if they exceed predefined thresholds, indicating a need for adaptation. For example, the CPU usage of the order management service has reached 90%, and the average response time has exceeded 300 ms. In the *Plan* phase, the system proposes a strategy based on pre-established rules. Typical examples would be to increase the number of replicas for overloaded microservices, or implement request rate limiting for non-critical features to prioritize essential operations. Next, in the *Execute* phase, the orchestrator applies the chosen strategy.

*Adaptation with* AWARE. The *Assess* agents continuously observe microservice metrics in real-time. Facing a situation similar to the one described for MAPE-K, characterized by increasing load and latency, the *Assess* agents infer that the traffic spike is caused by the promotional event. This inference is made either through contextual awareness (where information about the event is provided by the *Enrich* agents), or by leveraging predictive models trained on historical data. Based on this analysis, the agents determine that the system is overloaded and that adaptation is required to prevent performance degradation or system failure. They relay this information, including details of the event and its cause, to both the *Weigh* and *Reflect* agents for further action and analysis. The *Weigh* agents then determine several adaptation options to address the identified overload, such as the ones aforementioned for MAPE-K. Each option is evaluated based on various criteria, such as cost, impact on performance, and associated risks. After collaborative evaluation, the most suitable option is selected. Selected adaptation options are applied in a coordinated manner by the *Act* agents, ensuring that adaptations are executed correctly. Synchronization mechanisms between agents are employed to avoid conflicts, such as multiple agents trying to modify the same parameters simultaneously. Once the adaptations are complete, the *Reflect* agents immediately analyze their impact on system metrics. Insights from this analysis are integrated into the agents' knowledge bases to improve their preparedness for similar events in the future. For instance, a reward mechanism provides feedback to the *Weigh* agents to improve future generations and selections of adaptation options.

In this scenario, the AWARE framework overcomes the simple reactive adaptation of MAPE-K by leveraging *(i)* its contextual understanding (via *Assess* agents), *(ii)* its decision based on multiple criteria (via *Weigh* agents), and *(iii)* its anticipation and proactivity (via *Reflect* and *Enrich* agents). While MAPE-K applies a strategy based on predefined rules, AWARE can contextualize its environment, smartly compare several solutions, and take advantage of experiences to facilitate the future adaptation process.

## 4 Conclusion

The AWARE framework represents an improvement on MAPE-K, responding to the challenges of modern systems more efficiently and intelligently. While MAPE-K focuses on sequential cycles with centralized, monolithic knowledge, AWARE introduces agents capable of collaborating, learning continuously, and acting proactively. This approach makes it possible to react to changes and also to anticipate and adapt to them. AWARE highlights contextual perception, collective reasoning, and continuous learning. Each phase of AWARE contributes to making systems more responsive, resilient, and scalable. This distributed architecture, based on collaboration between agents, opens the way to a new generation of self-adaptive systems, better equipped to meet the demands of complex, dynamic environments. In short, AWARE does more than simply overcome the limitations of MAPE-K. It redefines what we can expect from a self-adaptive system: distributed intelligence, capable of improving over time and meeting the challenges of emerging technologies.

## Acknowledgments

This work was supported by a French government grant managed by the Agence Nationale de la Recherche under the France 2030 program, reference "ANR-23-PECL-0008".

## References


[1] Imen Abdennadher. 2022. DAACS: a decision approach for autonomic computing systems. *The Journal of Supercomputing* 78, 3 (2022), 3883–3904.

[2] Riadh Ben Halima, Marwa Hachicha, Ahmed Jemal, and Ahmed Hadj Kacem. 2023. MAPE-K patterns for self-adaptation in cyber-physical systems. *The Journal of Supercomputing* 79, 5 (2023), 4917–4943.

[3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[4] Jane Cleland-Huang, Ankit Agrawal, Michael Vierhauser, Michael Murphy, and Mike Prieto. 2022. Extending MAPE-K to support human-machine teaming. In *Proceedings of the 17th Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 120–131.

[5] Jacob Devlin. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[6] Simon Dobson, Spyros Denazis, Antonio Fernández, Dominique Gaïti, Erol Gelenbe, Fabio Massacci, et al. 2006. A survey of autonomic communications. *ACM Transactions on Autonomous and Adaptive Systems* 1, 2 (2006), 223–259.

[7] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, et al. 2023. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378* (2023).

[8] Zane Durante, Qiuyuan Huang, Naoki Wake, Ran Gong, et al. 2024. Agent AI: Surveying the Horizons of Multimodal Interaction. arXiv:2401.03568 [cs.AI]

[9] Ibrahim Elgendi, Md Farhad Hossain, Abbas Jamalipour, and Kumudu S Munasinghe. 2019. Protecting cyber physical systems using a learned MAPE-K model. *IEEE Access* 7 (2019), 90954–90963.

[10] Reinout Eyckerman, Phillipe Reiter, Siegfried Mercelis, Steven Latré, Johann Marquez-Barja, and Peter Hellinckx. 2021. A Generalized Approach For Practical Task Allocation Using A MAPE-K Control Loop. In *International Conference on Information and Communication Technology Convergence*. IEEE, 779–784.

[11] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2020. Generative adversarial networks. *Commun. ACM* 63, 11 (2020), 139–144.

[12] Ahmed E. Hassan, Gustavo Ansaldi Oliva, Dayi Lin, Boyuan Chen, and Zhen Ming Jiang. 2024. Rethinking Software Engineering in the Foundation Model Era: From Task-Driven AI Copilots to Goal-Driven AI Pair Programmers. *arXiv.org* abs/2404.10225 (2024). doi:10.48550/arxiv.2404.10225

[13] Rishi Hazra, Pedro Zuidberg Dos Martires, and Luc De Raedt. 2024. Saycanpay: Heuristic planning with large language models using learnable domain knowledge. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 20123–20133.

[14] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. *Advances in neural information processing systems* 33 (2020), 6840–6851.

[15] William Hunt, Toby Godfrey, and Mohammad D Soorati. 2024. Conversational Language Models for Human-in-the-Loop Multi-Robot Coordination. *arXiv preprint arXiv:2402.19166* (2024).

[16] Sharmin Jahan, Ian Riley, Charles Walter, Rose F Gamble, et al. 2020. MAPE-K/MAPE-SAC: An interaction framework for adaptive systems with security assurance cases. *Future Generation Computer Systems* 109 (2020), 197–209.

[17] Saeid Jamshidi, Ashkan Amirnia, Amin Nikanjam, and Foutse Khomh. 2024. Enhancing security and energy efficiency of cyber-physical systems using deep reinforcement learning. *Procedia Computer Science* 238 (2024), 1074–1079.

[18] Hendrik Jilderda and Claudia Raibulet. 2023. MAPE-K Based Guidelines for Designing Reactive and Proactive Self-adaptive Systems. In *Software Architecture. ECSA 2023 Tracks, Workshops, and Doctoral Symposium*. 53–68.

[19] Joao Paulo Karol Santos Nunes, Shiva Nejati, Mehrdad Sabetzadeh, and Elisa Yumi Nakagawa. 2024. Self-adaptive, Requirements-driven Autoscaling of Microservices. In *Proceedings of the 19th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 168–174.

[20] Jeffrey O Kephart and David M Chess. 2003. The vision of autonomic computing. *Computer* 36, 1 (2003), 41–50.

[21] Diederik P Kingma. 2013. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114* (2013).

[22] Christian Krupitzer, Felix Maximilian Roth, Sebastian VanSyckel, Gregor Schiele, and Christian Becker. 2015. A survey on engineering approaches for self-adaptive systems. *Pervasive and Mobile Computing* 17 (2015), 184–206.

[23] Evangelina Lara, Leocundo Aguilar, Mauricio A Sanchez, and Jesús A García. 2019. Adaptive security based on mape-k: A survey. *Applied Decision-Making: Applications in Computer Sciences and Engineering* (2019), 157–183.

[24] Sabah Lecheheb, Soufiane Boulehouache, and Said Brahimi. 2022. Improving Self-Adaptation by Combining MAPE-K, Machine and Deep Learning. In *2nd International Conference on New Technologies of Information and Communication*. IEEE, 1–6.

[25] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, and Michael R Lyu. 2023. Maat: Performance metric anomaly anticipation for cloud services with conditional diffusion. In *38th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 116–128.

[26] Jialong Li et al. 2024. Exploring the Potential of Large Language Models in Self-adaptive Systems. In *Proceedings of the 19th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 77–83.

[27] Jialong Li, Mingyue Zhang, Nianyu Li, Danny Weyns, Zhi Jin, and Kenji Tei. 2024. Generative AI for Self-Adaptive Systems: State of the Art and Research Roadmap. *ACM Trans. Auton. Adapt. Syst.* (2024). doi:10.1145/3686803

[28] Xianchang Luo, Yinxing Xue, Zhenchang Xing, and Jiamou Sun. 2022. Prcbert: Prompt learning for requirement classification using bert-based pretrained language models. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.

[29] Lukas Malburg, Maximilian Hoffmann, and Ralph Bergmann. 2023. Applying MAPE-K control loops for adaptive workflow management in smart factories. *Journal of Intelligent Information Systems* 61, 1 (2023), 83–111.

[30] Michael Maurer, Ivan Breskovic, Vincent C Emeakaroha, and Ivona Brandic. 2011. Revealing the MAPE loop for the autonomic management of cloud infrastructures. In *IEEE symposium on computers and communications*. IEEE, 147–152.

[31] Andreas Metzger, Clément Quinton, Zoltán Á. Mann, Luciano Baresi, and Klaus Pohl. 2024. Realizing self-adaptive systems via online reinforcement learning and feature-model-guided exploration. *Computing* 106, 4 (2024), 1251–1272.

[32] Hiroyuki Nakagawa and Shinichi Honiden. 2023. Mape-k loop-based goal model generation using generative ai. In *IEEE 31st International Requirements Engineering Conference Workshops*. IEEE, 247–251.

[33] Jiyoung Oh, Claudia Raibulet, and Joran Leest. 2022. Analysis of MAPE-K loop in self-adaptive systems for cloud, IoT and CPS. In *International Conference on Service-Oriented Computing*. Springer, 130–141.

[34] Clément Quinton, Michael Vierhauser, Rick Rabiser, Luciano Baresi, Paul Grünbacher, and Christian Schuhmayer. 2021. Evolution in dynamic software product lines. *Journal of software: evolution and process* 33, 2 (2021), e2293.

[35] Michael Riegler, Johannes Sametinger, and Michael Vierhauser. 2023. A distributed MAPE-K framework for self-protective IoT devices. In *18th Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 202–208.

[36] Célian Ringwald. 2024. Learning Pattern-Based Extractors from Natural Language and Knowledge Graphs: Applying Large Language Models to Wikipedia and Linked Open Data. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 23411–23412.

[37] Stuart J Russell and Peter Norvig. 2016. *Artificial intelligence: a modern approach*. Pearson.

[38] Eric Rutten, Nicolas Marchand, and Daniel Simon. 2017. Feedback control as MAPE-K loop in autonomic computing. In *Software Engineering for Self-Adaptive Systems III. Assurances: International Seminar, Dagstuhl Castle, Germany, December 15-19, 2013, Revised Selected and Invited Papers*. Springer, 349–373.

[39] Komal Sarda, Zakeya Namrud, Marin Litoiu, Larisa Shwartz, and Ian Watts. 2024. Leveraging Large Language Models for the Auto-remediation of Microservice Applications: An Experimental Study. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 358–369.

[40] Ishika Singh, Gargi Singh, and Ashutosh Modi. 2021. Pre-trained language models as prior knowledge for playing text-based games. *arXiv preprint arXiv:2107.08408* (2021).

[41] Jiyoung Song, Jeehoon Kang, Sangwon Hyun, Eunkyoung Jee, and Doo-Hwan Bae. 2022. Continuous verification of system of systems with collaborative MAPE-K pattern and probability model slicing. *Information and Software Technology* 147 (2022), 106904.

[42] Klaus Spremann. 1987. Agent and principal. In *Agency theory, information, and incentives*. Springer, 3–37.

[43] Marco Stadler, Johannes Sametinger, and Michael Riegler. 2024. Cyber-resilient edge computing: a holistic approach with multi-level MAPE-K loops. In *IEEE 21st International Conference on Software Architecture Companion*. 79–83.

[44] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).

[45] Steven Vere and Timothy Bickmore. 1990. A basic agent. *Computational intelligence* 6, 1 (1990), 41–60.

[46] Joakim Von Kistowski et al. 2018. Teastore: A micro-service reference application for benchmarking, modeling and resource management research. In *IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems*. IEEE, 223–236.

[47] Danny Weyns. 2020. *An introduction to self-adaptive systems: A contemporary software engineering perspective*. John Wiley & Sons.

[48] Danny Weyns, Sam Malek, and Jesper Andersson. 2012. FORMS: Unifying reference model for formal specification of distributed self-adaptive systems. *ACM Transactions on Autonomous and Adaptive Systems* 7, 1 (2012), 1–61.

[49] Danny Weyns, Bradley Schmerl, Vincenzo Grassi, Sam Malek, Raffaela Mirandola, Christian Prehofer, Jochen Wuttke, Jesper Andersson, Holger Giese, and Karl M Göschka. 2013. On patterns for decentralized control in self-adaptive systems. In *Software Engineering for Self-Adaptive Systems II: International Seminar, Dagstuhl Castle, Germany, October 24-29, 2010*. Springer, 76–107.